

Wireless Communications Homework I

Chao-Yang Zhan

October 5, 2026

Exercise 1 (*Erlang-B Implementation and Traffic Calculation*)

Trunking efficiency (η_T) depends on the channel assignment scheme and represents a trade-off between Quality of Service (QoS). It is fundamentally based on the **Erlang-B formula**, which computes the blocking probability $B(\rho, m)$ for a circuit-switched system with m channels and total offered traffic:

$$B(\rho, m) = \frac{\rho^m}{m! \sum_{k=0}^m \frac{\rho^k}{k!}} \quad (1)$$

where the parameters are defined as follows:

- m : the total number of channels in the trunk.
- $\rho = \lambda\mu$: the total offered traffic, where:
 - λ : the call arrival rate (Poisson call arrivals).
 - μ : the mean call duration (exponentially distributed).
- $G_c = \rho/m$: offered traffic per channel.

To implement this in MATLAB and find the total offered traffic load for specific blocking rates (1%, 3%, 5%, 10%), we utilize a robust recursive formulation to avoid numerical overflow.

Core Principle of the Recursive Formulation:

If we directly calculate the original Erlang-B formula:

$$B(\rho, m) = \frac{\frac{\rho^m}{m!}}{\sum_{k=0}^m \frac{\rho^k}{k!}} \quad (2)$$

when the number of channels m is large (e.g., 200 or more as required in this assignment), the factorial $m!$ and ρ^m become extremely large, leading to numerical overflow in the computer (resulting in NaN).

Recursive Reciprocal Formula:

To overcome this, we adopt an equivalent and numerically stable recurrence relation:

$$\frac{1}{B(\rho, m)} = 1 + \frac{m}{\rho} \cdot \frac{1}{B(\rho, m-1)} \quad (3)$$

By iterating through a loop from $k = 1$ to m , we progressively compute the reciprocal value `invB` (i.e., $\frac{1}{B(\rho, m)}$).

Matlab Implementation:

Finally, taking the reciprocal of `invB` ($1.0/\text{invB}$), we can safely and accurately obtain the final blocking probability B .

```
clear; clc; format long g;

% Recursive Erlang-B calculation
function B = erlangB(rho, m)
    if rho <= 0
        B = (m == 0);
        return;
    end
    invB = 1.0;
    for k = 1:m
        invB = 1.0 + (k / rho) * invB;
    end
    B = 1.0 / invB;
end

% Set channel number range (1~20 and 200~220)
m_range = [1:20, 200:220];
blocking_rates = [0.01, 0.03, 0.05, 0.10];

% Loop through each channel number and solve for offered traffic rho

fprintf('Channel (m) | 1%% Blocking | 3%% Blocking | 5%%\n');
        Blocking | 10%% Blocking\n');

for i = 1:length(m_range)
    m = m_range(i);

    rho_1 = fzero(@(rho) erlangB(rho, m) - 0.01, [0, max(1, m * 10)]);
    rho_3 = fzero(@(rho) erlangB(rho, m) - 0.03, [0, max(1, m * 10)]);
    rho_5 = fzero(@(rho) erlangB(rho, m) - 0.05, [0, max(1, m * 10)]);
    rho_10 = fzero(@(rho) erlangB(rho, m) - 0.10, [0, max(1, m * 10)]);

    % Print results in table format
    fprintf('%11d | %12.4f | %12.4f | %12.4f | %12.4f\n', ...
        m, rho_1, rho_3, rho_5, rho_10);
end
```

Listing 1: Matlab implementation of Erlang-B and traffic solver

Simulation Results:

The resulting offered traffic loads for each channel count under different blocking probability constraints are displayed in the MATLAB Command Window below:

Command Window				
Channel (m)	1% Blocking	3% Blocking	5% Blocking	10% Blocking
1	0.0101	0.0309	0.0526	0.1111
2	0.1526	0.2816	0.3813	0.5954
3	0.4555	0.7151	0.8994	1.2708
4	0.8694	1.2589	1.5246	2.0454
5	1.3608	1.8752	2.2185	2.8811
6	1.9090	2.5431	2.9603	3.7584
7	2.5009	3.2497	3.7378	4.6662
8	3.1276	3.9865	4.5430	5.5971
9	3.7825	4.7479	5.3702	6.5464
10	4.4612	5.5294	6.2157	7.5106
11	5.1599	6.3280	7.0764	8.4871
12	5.8760	7.1410	7.9501	9.4740
13	6.6072	7.9667	8.8349	10.4699
14	7.3517	8.8035	9.7295	11.4735
15	8.1080	9.6500	10.6327	12.4838
16	8.8750	10.5052	11.5436	13.5001
17	9.6516	11.3683	12.4613	14.5217
18	10.4369	12.2384	13.3852	15.5480
19	11.2301	13.1150	14.3147	16.5787
20	12.0306	13.9974	15.2493	17.6132
200	179.7380	190.8859	198.5073	214.3226
201	180.7059	191.8943	199.5456	215.4278
202	181.6739	192.9028	200.5839	216.5331
203	182.6420	193.9114	201.6224	217.6383
204	183.6103	194.9201	202.6609	218.7437
205	184.5787	195.9289	203.6994	219.8490
206	185.5473	196.9378	204.7381	220.9544
207	186.5161	197.9468	205.7768	222.0598
208	187.4850	198.9559	206.8156	223.1653
209	188.4540	199.9651	207.8544	224.2708
210	189.4232	200.9744	208.8933	225.3763
211	190.3925	201.9837	209.9323	226.4818
212	191.3620	202.9932	210.9714	227.5874
213	192.3316	204.0028	212.0105	228.6931
214	193.3013	205.0124	213.0497	229.7987
215	194.2712	206.0222	214.0889	230.9044
216	195.2412	207.0320	215.1283	232.0102
217	196.2114	208.0419	216.1676	233.1159
218	197.1816	209.0519	217.2071	234.2217
219	198.1521	210.0620	218.2466	235.3275
220	199.1226	211.0722	219.2862	236.4334

Figure 1: Offered traffic solutions for $m = 1 \sim 20$ and $m = 200 \sim 220$ at 1%, 3%, 5%, and 10% blocking rates.

Exercise 2 (*Analysis of Offered Traffic and Carried Traffic*)

(a) Could the total offered traffic load be larger than the number of available channels?

Answer and Explanation:

Yes, absolutely. Think of offered traffic ($\rho = \lambda\mu$) as the **total demand or people wanting to make calls**, while the number of channels (m) is just the physical limit of phone lines you actually have. In real-world systems, we always allow a little bit of blocking (like 1% to 10%). When everyone tries to call at once during busy hours, the total demand (ρ) easily goes way above the physical channel limit (m), and the system simply runs at its maximum capacity while blocking the extra callers.

(b) How to determine the actual traffic that has been served?

Solution:

The actual traffic successfully handled by the system (Carried Traffic, A_c) is simply the total offered traffic multiplied by the probability that a call goes through (i.e., not being blocked):

$$A_c = \rho \cdot (1 - B(\rho, m)) \quad (4)$$

Alternatively, you can look at it from a statistics point of view: it's just the average number of busy channels in steady-state operation:

$$A_c = \sum_{k=0}^m k \cdot P_k \quad (5)$$

where P_k denotes the probability that exactly k channels are busy in steady state (and specifically, $P_m = B(\rho, m)$ represents the blocking probability when all channels are fully occupied).

Exercise 3 (*Frequency Reuse and Spectral Efficiency*)

Given a total of 600 channels shared equally among 1, 2, or 3 operators using a frequency reuse factor $N = 5$.

(a) **Maximum Offered Traffic Load per Cell:**

Depending on the number of operators, the number of channels per cell C_{cell} varies:

- **1 Operator:** $C_{\text{cell}} = \frac{600/1}{5} = 120$ channels.
- **2 Operators:** $C_{\text{cell}} = \frac{600/2}{5} = 60$ channels.
- **3 Operators:** $C_{\text{cell}} = \frac{600/3}{5} = 40$ channels.

By substituting these values into the Erlang-B formula for each target blocking rate (1%, 3%, 5%, 10%), we obtain the corresponding maximum offered traffic per cell.

Simulation Results for Exercise 3:

The maximum offered traffic load per cell for each operator scenario under different blocking probabilities is shown below:

Operators	Channels/Cell (C_{cell})	1% Blocking	3% Blocking	5% Blocking	10% Blocking
1	120.00	102.9636	110.6506	115.7706	126.0824
2	60.00	46.9497	51.5698	54.5656	60.4013
3	40.00	29.0074	32.4118	34.5960	38.7874

Figure 2: Maximum offered traffic per cell for 1, 2, and 3 operators at various blocking rates.

(b) Which case is more spectral efficient?

Case 1 (one operator) is the most spectrally efficient.

Reasoning and G_c Calculation:

According to system theory, spectral efficiency can be written as the product of bandwidth efficiency (η_B), spatial efficiency (η_C), and trunking efficiency (η_T):

$$\eta_s = \underbrace{\frac{1}{W_c}}_{\eta_B} \times \underbrace{\frac{1}{N \times A}}_{\eta_C} \times \underbrace{G_c}_{\eta_T} \quad (6)$$

where $G_c = \frac{\rho}{C_{\text{cell}}}$ represents the offered traffic per channel (trunking efficiency, η_T).

Since channel bandwidth W_c , reuse factor $N = 5$, and cell area A are constant across all cases, the spatial and bandwidth efficiency terms (η_B and η_C) are identical. Therefore, **the overall spectral efficiency η_s is directly proportional to G_c .**

Using the total offered traffic ρ from our simulation results, we compute the offered traffic per channel ($G_c = \frac{\rho}{C_{\text{cell}}}$) for each case across all blocking rates:

• **1 Operator ($C_{\text{cell}} = 120$):**

- 1% Blocking: $G_c = \frac{102.9636}{120} \approx 0.8580$ Erlangs/channel
- 3% Blocking: $G_c = \frac{110.6506}{120} \approx 0.9221$ Erlangs/channel
- 5% Blocking: $G_c = \frac{115.7706}{120} \approx 0.9648$ Erlangs/channel
- 10% Blocking: $G_c = \frac{126.0824}{120} \approx 1.0507$ Erlangs/channel

• **2 Operators ($C_{\text{cell}} = 60$):**

- 1% Blocking: $G_c = \frac{46.9497}{60} \approx 0.7825$ Erlangs/channel
- 3% Blocking: $G_c = \frac{51.5698}{60} \approx 0.8595$ Erlangs/channel
- 5% Blocking: $G_c = \frac{54.5656}{60} \approx 0.9094$ Erlangs/channel
- 10% Blocking: $G_c = \frac{60.4013}{60} \approx 1.0067$ Erlangs/channel

• **3 Operators ($C_{\text{cell}} = 40$):**

- 1% Blocking: $G_c = \frac{29.0074}{40} \approx 0.7252$ Erlangs/channel
- 3% Blocking: $G_c = \frac{32.4118}{40} \approx 0.8103$ Erlangs/channel
- 5% Blocking: $G_c = \frac{34.5960}{40} \approx 0.8649$ Erlangs/channel
- 10% Blocking: $G_c = \frac{38.7874}{40} \approx 0.9697$ Erlangs/channel

As shown above, pooling all 600 channels under a single operator yields a larger trunk size ($C_{\text{cell}} = 120$), which consistently provides higher G_c values at any given blocking probability. Consequently, **Case 1 achieves the highest trunking efficiency (G_c) and thus the highest spectral efficiency (η_s).**